

Workshop Walk Through

Delta — AI-Powered LED Circuit Guide

Presented by Eyob, Tia & Christian | Georgia Gwinnett College — STEC 4800

What is Project Impulse?

Project Impulse was developed to reduce the loss of autonomous robots in hazardous environments. Using Delta — our AI controller — the system adjusts hardware output in real time. In this workshop, you will build a simplified version of that system: an AI-controlled LED circuit.

Workshop Overview — 3 Phases

01 Circuit Building

Build your LED circuit using an Arduino UNO, breadboard, resistor, jumper wires, and LED.

02 AI Calibration

Customize Delta with your name and fun facts, then set it up on your computer.

03 LED Code & Test

Ask Delta to control the light — blink fast, blink slow, turn on, turn off.

What is Delta?

Delta is the AI stand-in for Theta (Project Impulse's real controller). Delta specializes in situational probability and uses your LED to express its 'Proceed' recommendation.

01

Circuit Building — Parts List

Before you begin, gather all of the components below. Everything is required to complete the circuit.

Component	Qty	Purpose
Arduino UNO	1x	The microcontroller — the brain of the circuit
Breadboard	1x	Platform for connecting components without soldering
LED	1x	Light Emitting Diode — the light Delta will control
Resistor (220Ω)	1x	Limits current to protect the LED from burning out
Jumper Wires	2x	Male-to-male wires to connect components

No soldering required!

All connections are made by inserting jumper wires into the breadboard and Arduino pin headers.

How the Circuit Works

The Arduino sends a HIGH or LOW signal out of GPIO Pin 11. When HIGH, current flows from Pin 11 through the resistor and into the LED's positive leg (anode), lighting it up. When LOW, the current stops and the LED turns off. The resistor (220Ω) sits in between to limit current and protect the LED.

The negative leg (cathode) of the LED connects back to the GND pin on the Arduino, completing the circuit.

Wiring the Circuit — Step by Step

1. Place your LED into the breadboard. The longer leg is the positive leg (anode). Note which row each leg is in.
2. Insert one end of the 220Ω resistor into the same breadboard row as the LED's positive (longer) leg.
3. Connect a jumper wire from the other end of the resistor to GPIO Pin 11 on the Arduino.
4. Connect a second jumper wire from the LED's negative (shorter) leg to the breadboard's ground (–) rail.
5. Connect a jumper wire from the breadboard's ground (–) rail to a GND pin on the Arduino.
6. Plug the Arduino into your computer via USB. The power LED on the Arduino should light up.
7. Double-check all connections before moving!

Arduino UNO — Pin Reference

The diagram below shows the full Arduino UNO board. The three highlighted pins are the ones used in this workshop:

Arduino Pin	Wire Color	Connects To
GND (Ground)	Black wire	Negative rail of breadboard → completes the circuit
GPIO Pin 11	Any color wire	LED positive leg (via resistor) → sends HIGH/LOW signal

Finding the pins on your board:

GND — Located in the power header section on the left side of the board, labeled 'GND'.
3.3V — Also in the power header, labeled '3.3V'. Not used in this circuit but good to know!
GPIO Pin 11 — On the right side digital pin strip, labeled '~11' (the ~ means it supports PWM).

02

AI Calibration — Setting Up Delta

Delta is the AI that will control your LED. Follow these steps to get it running on your computer.

Installing Delta from the ZIP File

8. Locate the Delta ZIP file provided by your workshop leader (or download from the shared GitHub link).
9. Right-click the ZIP file and select Extract All (Windows) or double-click to unzip (Mac).
10. Open the extracted Delta folder — you should see a file called run_windows inside.
11. To add to GitHub: go to your repository, click Add File > Upload Files, and drag the entire Delta folder in. Commit the changes.

GitHub Tip:

Upload the whole Delta folder as-is. Do not rename any files inside the folder or Delta may not run correctly.

Launching Delta

12. Open the Delta folder on your computer.
13. Double-click the run_windows file. A terminal window and the Delta interface will open.
14. You should see: "Delta: System initialized. Hardware status: OK."
15. If you see that message — Delta is ready! Move on to calibration.

Calibrating Delta (Personalizing Your AI)

Before controlling the light, introduce yourself to Delta so it remembers you:

- Type your name in the message box and tell Delta a couple of fun facts about yourself.
- Delta will remember these facts for the session — you can ask it to recall them later!
- Example: "Hi Delta, my name is Jordan. I love basketball and I'm studying IT."

03

LED Code — Ask Delta to Control the Light!

If everything is working correctly, then ask Delta to control the light!

With your circuit built and Delta running, you can now give Delta commands to control your LED. Try each one below:

Light Commands to Try

Command	What to Say to Delta	Expected Result
Turn Light On	"Turn the light on" or "Light on"	LED stays solid/on continuously
Turn Light Off	"Turn the light off" or "Light off"	LED turns off completely
Blink Light Fast	"Blink the light fast"	LED blinks rapidly — high frequency
Blink Light Slow	"Blink the light slow"	LED blinks slowly — low frequency

Your Turn — Explore!

Now that Delta responds to basic commands, explore further:

- Tell Delta: "Let's run a scenario" — watch how it responds!
- Ask Delta: "What is my name?" — it should remember from calibration.
- Ask it other personal facts you shared during calibration.
- Try giving Delta custom light instructions and see how it interprets them.

Troubleshooting

LED not lighting up?

Check that the longer LED leg connects (via resistor) to Pin 11, and the shorter leg connects to GND. Make sure the Arduino is plugged in via USB.

Delta won't open?

Make sure you extracted the ZIP file fully. Double-click run_windows — do not run from inside the ZIP.

LED blinking but wrong speed?

Rephrase your command to Delta. Try: "Blink the light as fast as possible" or "Blink slowly, one blink per second."

Arduino not recognized?

Try a different USB port. Ask a workshop leader — driver issues are common and easy to fix.

